UNIVERSIDAD POLITÉCNICA BICENTENARIO

Práctica 5: Monitor de Voltaje y Temperatura con ESP32

Presenta:

Jesús Ángel Bonilla Duran

Viridiana Santibáñez Carrillo

Jocelyn Rodriguez Torres

Matricula:

24033407

24033066

24033436

Asignatura:

Colocar Nombre de la Asignatura

Profesor:

Hugo Antonio Méndez Guzmán

Silao, Gto. a 06 de noviembre de 2026

Contenido

Introducción 3
Marco Teórico 3
Sensor de Temperatura LM35 3
¿Qué es el protocolo I²C? 4
Supervisión de datos por comunicación UART 4
¿Qué es UART en un sistema embebido? 4
¿Cómo funciona la comunicación UART? 4
Desarrollo 5
Material requerido 5
Explicación del Código empleado 6
Resultados 8
Observaciones y Conclusiones 10
Referencias 10

Lista de Figuras

Figura 1 IMAGEN WODKI 5
Figura 2. Montaje del circuito en protoboard con la ESP32, los potenciómetros y la pantalla LCD... 8
Figura 3 Prueba de funcionamiento del sistema con ajuste de los límites de temperatura mediante los potenciómetros. 9
Figura 4 Visualización de la temperatura medida y los límites establecidos durante el funcionamiento del sistema. 9
Figura 5. Prueba de funcionamiento del sistema de control de temperatura, mostrando los límites inferior y superior, la temperatura medida y el estado de calentamiento en la pantalla LCD. 10

Introducción

Los sistemas de automatización e instrumentación industrial requieren dispositivos capaces de medir, procesar y mostrar variables físicas del entorno. Entre estas variables se encuentra la temperatura, cuyo monitoreo es importante para supervisar el funcionamiento de equipos, controlar procesos y prevenir condiciones que puedan afectar el desempeño de un sistema.

En esta práctica se aborda el uso de una tarjeta ESP32 para realizar la adquisición y visualización de señales relacionadas con la medición de temperatura. Para ello, se considera el empleo del sensor LM35, el cual permite obtener una señal de voltaje analógica proporcional a la temperatura detectada. También se contempla el uso de potenciómetros como entradas analógicas y una pantalla LCD de 16×2 con comunicación I²C para mostrar la información obtenida.

Durante el desarrollo se estudia cómo la ESP32 recibe señales analógicas y las convierte en valores digitales mediante su convertidor analógico-digital (ADC). Asimismo, se revisa la relación entre el voltaje de salida del sensor y la temperatura, considerando que el LM35 proporciona una variación aproximada de 10 mV por cada grado Celsius.

La práctica permite comprender la importancia de los sensores, la adquisición de datos y la comunicación entre dispositivos electrónicos. Estos conocimientos representan una base para desarrollar sistemas de monitoreo e instrumentación que pueden aplicarse posteriormente en procesos de automatización y control industrial.

Marco Teórico

Sensor de Temperatura LM35

El LM35 es un sensor de temperatura popular reconocido por su facilidad de uso y precisión en la medición de la temperatura, que ofrece una respuesta eléctrica lineal directamente correlacionada con los cambios de temperatura en los grados Celsius (° C).A diferencia de los termopares y los termistores, el diseño del LM35 elimina la necesidad de una mayor calibración, lo que lo convierte en una opción adecuada para aplicaciones donde se valora el seguimiento de temperatura simple pero preciso.Esta discusión profundiza en los detalles del sensor de temperatura LM35, examinando su configuración de PIN, especificaciones técnicas, equivalentes alternativos, características distintivas y consejos prácticos sobre su aplicación efectiva en una variedad de proyectos electrónicos (Jason, s.f.).

¿Qué es el protocolo I²C?

I2C (Inter-Integrated Circuit) es un protocolo de comunicación serial, multi-maestro, multi-esclavo, de conmutación de paquetes, de terminación simple y de dos hilos, desarrollado por Philips Semiconductors (ahora NXP) en 1982. También se le conoce como IIC. Opera tanto en la capa de enlace de datos como en la capa física del modelo OSI, proporcionando un método estandarizado para la comunicación dentro de la placa entre circuitos integrados (Understanding I2C Protocol: essential for embedded systems in 2025, s.f.).

Supervisión de datos por comunicación UART

Reconocida por su simplicidad y eficiencia, la UART en sistemas embebidos desempeña un papel crucial al permitir un intercambio de datos fluido entre dispositivos electrónicos.

Como protocolo de comunicación en serie, UART (Transmisor/Receptor Asíncrono Universal) facilita la comunicación asíncrona, permitiendo que dispositivos con diferentes frecuencias de reloj se comuniquen sin necesidad de una señal de reloj compartida.

Al convertir los datos paralelos a formato serial para su transmisión y viceversa para su recepción, la UART garantiza una comunicación fiable y sencilla, lo que la hace indispensable en diversas aplicaciones integradas.

¿Qué es UART en un sistema embebido?

En esencia, UART es un protocolo de comunicación en serie que permite el intercambio de datos entre dispositivos electrónicos, se trata de un canal para la comunicación asíncrona, que permite que dispositivos con frecuencias de reloj o estructuras contrastantes se comuniquen sin problemas.

El término "asíncrono" en UART significa que no tiene una señal de reloj compartida para sincronizar los bits de salida desde la unidad transmisora al extremo receptor. En cambio, la transferencia de datos se produce bit a bit, y el receptor y el transmisor deben operar a la misma velocidad en baudios, la velocidad a la que se transfiere la información, para garantizar una comunicación precisa en la comunicación UART, dos UART se comunican directamente entre sí.

¿Cómo funciona la comunicación UART?

La comunicación UART comienza con la configuración de los dispositivos con los mismos parámetros: velocidad de transmisión, bits de datos, paridad y bits de parada, una vez inicializado, el remitente inicia la comunicación transmitiendo un "bit de inicio" (0 lógico).

Esto indica el comienzo de una trama de datos y ayuda al receptor a sincronizar su reloj con los datos entrantes. Los bits de datos propiamente dichos se transmiten después del bit de inicio, si está configurado, se incluye un bit de paridad después de los bits de datos para la comprobación de errores.

Desarrollo

Material requerido

- ESP32 DevKit V1.
- Sensor de temperatura LM35.
- Potenciómetro 1.
- Potenciómetro 2.
- Pantalla LCD de 16×2 con módulo I²C.
- Protoboard.
- Cables de conexión.
- Fuente de alimentación adecuada.
- Computadora.
- Visual Studio Code.
- PlatformIO.
- Wokwi.

Como primera parte se identificaron los componentes necesarios para el desarrollo de la práctica, considerando la ESP32 como unidad principal de procesamiento, el sensor LM35 como elemento de medición de temperatura, los potenciómetros como entradas analógicas y la pantalla LCD como medio de visualización.

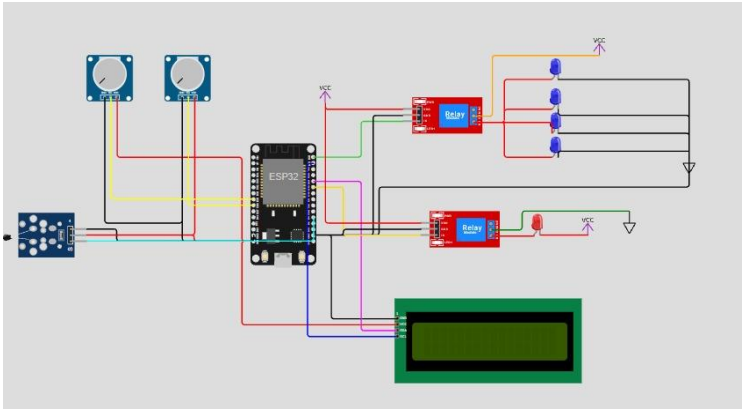


Figura 1 IMAGEN WODKI

A continuación, se presenta el código empleado

Main.cpp

```
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <OneWire.h>
#include <DallasTemperature.h>

#define PIN_DS18B20 4

#define FOCO_CALENTAR 19
#define ENFRIAMIENTO  23

#define POT_LL 25
#define POT_HL 26

int HL = 35;
float ema_LL = 0;
float ema_HL = 0;
const float alpha = 0.1;

LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
OneWire oneWire(PIN_DS18B20);
DallasTemperature sensors(&oneWire);

bool enfriando = false;
unsigned long tiempoLectura = 0;
unsigned long tiempoLCD     = 0;
float temperatura = 25.0;

void setup() {
  pinMode(FOCO_CALENTAR, OUTPUT);
  pinMode(ENFRIAMIENTO, OUTPUT);

  digitalWrite(FOCO_CALENTAR, HIGH);
  digitalWrite(ENFRIAMIENTO, HIGH);

  sensors.begin();
  sensors.setResolution(11);
  sensors.setWaitForConversion(false);
  sensors.requestTemperatures();

  Wire.begin(21, 22);
  lcd.init();
  lcd.backlight();
  lcd.clear();
  ema_LL = analogRead(POT_LL);
  ema_HL = analogRead(POT_HL);
}

void loop() {
  ema_LL = (alpha * analogRead(POT_LL)) + ((1.0 - alpha) * ema_LL);
  ema_HL = (alpha * analogRead(POT_HL)) + ((1.0 - alpha) * ema_HL);

  LL = map((int)ema_LL, 0, 4095, 0, 60);
  HL = map((int)ema_HL, 0, 4095, 0, 60);

  if (LL >= HL) {
    LL = HL - 1;
    if (LL < 0) {
      LL = 0;
      HL = 1;
    }
  }

  if (millis() - tiempoLectura >= 375) {
    tiempoLectura = millis();

    float tempLeida = sensors.getTempCByIndex(0);
    if (tempLeida != DEVICE_DISCONNECTED_C && tempLeida > -50.0) {
      temperatura = tempLeida;
    }
    sensors.requestTemperatures();
    if (temperatura >= HL) {
      enfriando = true;
    } else if (temperatura <= LL) {
      enfriando = false;
    }
    if (enfriando) {
      digitalWrite(FOCO_CALENTAR, HIGH);
      digitalWrite(ENFRIAMIENTO, LOW);
    } else {
      digitalWrite(ENFRIAMIENTO, HIGH);
      digitalWrite(FOCO_CALENTAR, LOW);
    }
  }

  if (millis() - tiempoLCD >= 150) {
    tiempoLCD = millis();
    char buffer[10];
    lcd.setCursor(0, 0);
    sprintf(buffer, "LL=%-2dC ", LL);
    lcd.print(buffer);

    lcd.setCursor(8, 0);
    dtostrf(temperatura, 4, 1, buffer);
    lcd.print("T=");
    lcd.print(buffer);
    lcd.print("C ");
    lcd.setCursor(0, 1);
    sprintf(buffer, "HL=%-2dC ", HL);
    lcd.print(buffer);

    lcd.setCursor(8, 1);
    if (enfriando) {
      lcd.print("[ENFR.] ");
    } else {
      lcd.print("[CALEN] ");
    }
  }
}
```

Explicación del Código empleado

Librerías:

Enseguida se preparó el entorno de programación en Visual Studio Code mediante la extensión PlatformIO, donde se trabaja con el lenguaje C++ y las bibliotecas necesarias para utilizar la pantalla LCD y las funciones de la tarjeta ESP32 , en la primera parte del código se incluyen las librerías necesarias para el funcionamiento del sistema de arduino,

desde la mas básica como establecer la comunicación I2C, ya que facilita el control de la pantalla LCD para mostrar información.y asi obtener las lecturas de temperatura del sensor.

Declaración de pines:

En esta parte se definen los pines de la ESP32 que se utilizan para conectar los componentes del circuito. Se establecen las salidas para el calentamiento y enfriamiento, así como las entradas analógicas de los dos potenciómetros, que permiten ajustar los límites de temperatura.

Declaración de variables

Se declaran las variables necesarias para almacenar los límites de temperatura inferior y superior (LL y HL), así como los valores utilizados para realizar el filtrado de las lecturas analógicas.
También se establece la variable alpha, que determina el nivel de suavizado aplicado a las señales de los potenciómetros. La variable temperatura almacena la lectura obtenida por el sensor y enfriando permite identificar el estado de funcionamiento del sistema.

Configuración inicial `(void setup)`

En la función `setup()` se realiza la configuración inicial del sistema. Se establecen los pines de calentamiento y enfriamiento como salidas mediante `pinMode()`.
Posteriormente, se inicializa el sensor de temperatura y la comunicación I2C para utilizar la pantalla LCD. Finalmente, se activa la iluminación de la pantalla y se realizan las primeras lecturas de los potenciómetros.

Lectura de los potenciómetros

Dentro de la función `loop()` se obtienen continuamente las lecturas analógicas de los dos potenciómetros mediante `analogRead()`.
Los valores obtenidos se procesan mediante un filtro exponencial, con la finalidad de suavizar las variaciones de las señales. Después, la función `map()` convierte las lecturas del ADC, cuyo rango es de 0 a 4095, a valores de temperatura de 0 a 60 °C.

Establecimiento de límites de temperatura

Se incluye una condición para evitar que el límite inferior sea igual o mayor que el límite superior. Cuando esto sucede, el programa ajusta los valores para mantener una diferencia entre ambos límites y evitar conflictos durante el control del sistema.

Lectura de temperatura

El programa utiliza la función `millis()` para controlar el tiempo entre las lecturas de temperatura, estableciendo un intervalo de 375 milisegundos.
La temperatura obtenida se almacena en la variable correspondiente y se verifica que la lectura sea válida antes de actualizar su valor.

Control de calentamiento y enfriamiento

En esta parte se comparan los valores de temperatura medidos con los límites establecidos mediante los potenciómetros.

Cuando la temperatura alcanza o supera el límite superior, se activa el estado de enfriamiento. Por otro lado, cuando la temperatura desciende hasta el límite inferior, se desactiva el enfriamiento y se activa el calentamiento.
De esta manera, el sistema cambia automáticamente entre ambos estados dependiendo de la temperatura registrada.

Visualización en la pantalla LCD

Finalmente, se utiliza la pantalla LCD de 16×2 para mostrar la información del sistema.
En la primera línea se visualizan el límite inferior y la temperatura medida, mientras que en la segunda línea se muestra el límite superior y el estado actual del sistema, indicando si se encuentra en calentamiento o enfriamiento.
La actualización de la pantalla se realiza mediante la función `millis()`, estableciendo un intervalo de 150 milisegundos.

Resultados

Durante el desarrollo de la práctica se logró implementar el sistema de monitoreo y control de temperatura utilizando la tarjeta ESP32, el sensor de temperatura LM35, dos potenciómetros y una pantalla LCD de 16×2.

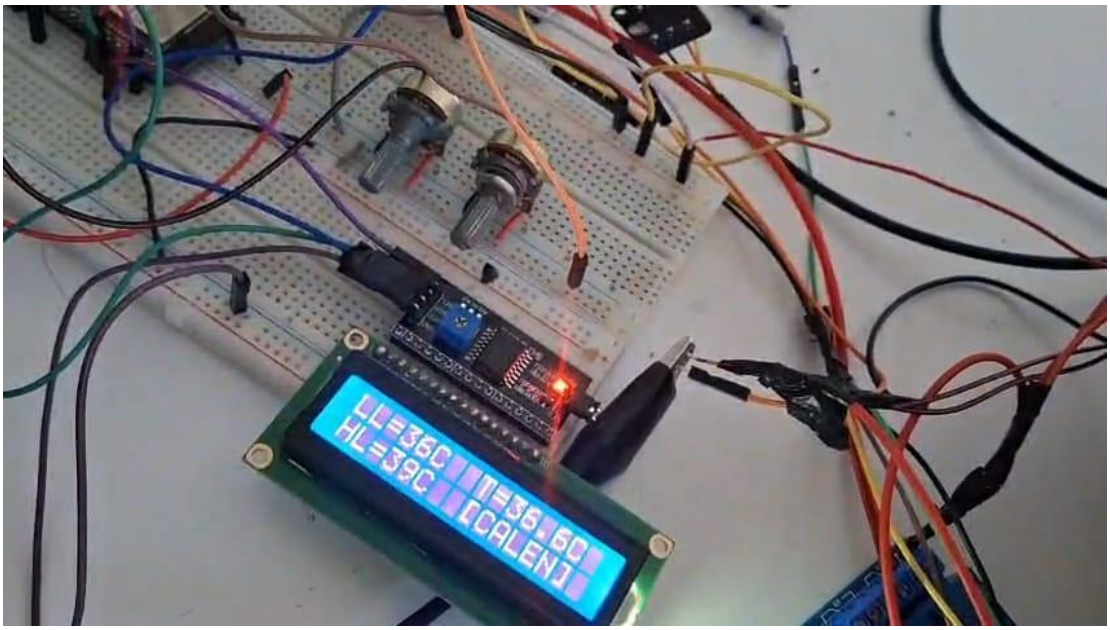


Figura 2. Montaje del circuito en protoboard con la ESP32, los potenciómetros y la pantalla LCD.

Como se observa en las evidencias, se realizaron las conexiones de los componentes en la protoboard y se verificó la visualización de los datos en la pantalla LCD, en ella se muestran la temperatura medida (T), el límite inferior (LL) y el límite superior (HL).

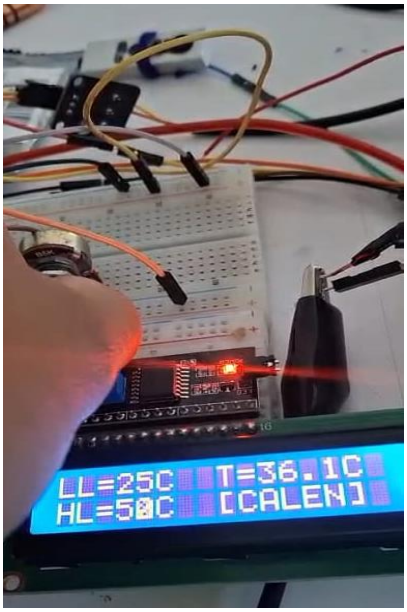


Figura 3 Prueba de funcionamiento del sistema con ajuste de los límites de temperatura mediante los potenciómetros.



Figura 4 Visualización de la temperatura medida y los límites establecidos durante el funcionamiento del sistema.

También se comprobó que, al modificar los potenciómetros, los límites de temperatura cambiaban y el sistema actualizaba su estado de funcionamiento, la pantalla permitió identificar cuándo se encontraba en modo de calentamiento (CALEN) y cuándo cambiaba al modo de enfriamiento (ENFR.).

En las pruebas realizadas se observaron diferentes valores de temperatura y límites de referencia, lo que permitió comprobar la actualización de las lecturas y la respuesta del sistema ante los cambios realizados mediante los potenciómetros.

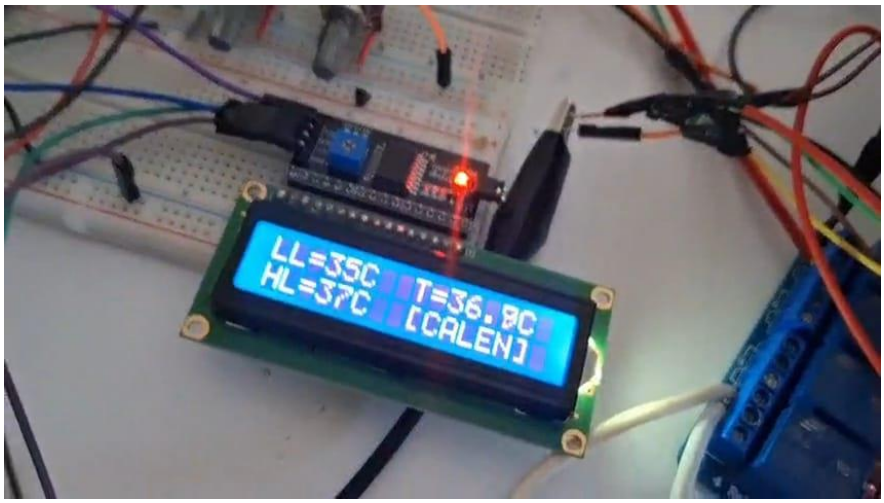


Figura 5. Prueba de funcionamiento del sistema de control de temperatura, mostrando los límites inferior y superior, la temperatura medida y el estado de calentamiento en la pantalla LCD.

Observaciones y Conclusiones

Durante el desarrollo de la práctica se observó cómo la ESP32 puede utilizarse para recibir y procesar señales analógicas provenientes de sensores y potenciómetros. También se comprobó la utilidad de la pantalla LCD para visualizar la temperatura y los límites establecidos, facilitando la supervisión del sistema.

Una de las observaciones más importantes fue que, al modificar los potenciómetros, los valores de los límites inferior y superior cambiaban, permitiendo ajustar las condiciones de funcionamiento. Además, se pudo identificar en la pantalla cuándo el sistema se encontraba en estado de calentamiento o enfriamiento.

En conclusión, esta práctica permitió reforzar los conocimientos sobre programación de microcontroladores, lectura de señales analógicas y comunicación I2C. También se comprendió cómo integrar diferentes componentes electrónicos para desarrollar un sistema de monitoreo y control de temperatura. Finalmente, se reconoció la importancia de establecer correctamente los límites de operación para lograr un funcionamiento adecuado del sistema.

Referencias

[1] Jason. (s.f.). Obtenido de Distribuidores de componentes electrónicos - componentes IC
[2] Smith, G. M. (5 de julio de 2021). ¿Qué es el acondicionamiento de señal o acondicionador de señal
[3] Understanding I2C Protocol: essential for embedded systems in 2025. (s.f.).